

5

Can I Pause This? – Application States

A piece of software like a video game is rarely as simple as the term suggests. Most of the time, it's not just the game mechanics and rendering one has to deal with in such an application. Nowadays, an industry-standard product also includes a nice introduction animation before the game begins. It also has a menu for the player to tinker with in order to start playing the game, manage the different settings it offers, view the credits or quit the application. On top of that, the title of this chapter also suggests the possibility of putting your game on pause for a moment or two. In hindsight, it's simple conveniences like this that draw the line in the sand between a game in its early stages, that's awkward to navigate and possibly confusing, and a product that offers the same level of control as most games on the market. To supply the backbone to such an idea, in this chapter we will be covering:

- Implementing the state manager
- Upgrading the event manager to handle different states
- Creating different states for the introduction, main menu and game-play sections of our game
- Providing the means to pause the game
- Implementing state blending
- Stringing the states together to create cohesive application flow

What is a state?

Before we get into any kind of implementation, it's necessary to understand what we're dealing with. If you've been reading up on any kind of game development material before, you probably came across the term **state**. It can mean different things, depending on its context. In this case, a state is any one of the many different layers of your game, like the main menu, the intro that plays before the menu is shown, or the actual game-play. Naturally, each one of these layers has its own way of updating itself and rendering its contents onto the screen. The game developer's job when utilizing this system is to break down a given problem into separate, manageable states and transitions between them. This essentially means that if you are presented with the problem of having a menu in the game, the solution would be creating two states, one for the menu and one for your game-play, and transitioning between the two at appropriate times.

The most simplistic approach

Let's begin by illustrating the most common approach newcomers take in order to solve this problem. It starts by enumerating all the possible states a game could have:

```
enum class StateType{
    Intro = 1, MainMenu, Game, Paused, GameOver, Credits
};
```

Good start. Now let's put it to work by simply using a switch statement:

```
void Game::Update() {
    switch(m_state) {
        case(StateType::Intro):
            UpdateIntro();
            break;
        case(StateType::Game):
            UpdateGame();
            break;
        case(StateType::MainMenu):
            UpdateMenu();
            break;
        ...
    }
}
```

The same goes for drawing it on screen:

```
void Game::Render() {
    switch(m_state) {
        case(StateType::Intro):
            DrawIntro();
            break;
        case(StateType::Game):
            DrawGame();
            break;
        case(StateType::MainMenu):
            DrawMenu();
            break;
        ...
    }
}
```

While this approach is okay for really small games, scalability here is completely out of the question. First of all, the switch statements are going to continue to grow when more states are added. Assuming we keep the functionality for updating and rendering a specific state localized to just one method, the number of these methods will also continue to grow by at least two methods per state, one of them being used for updating and another for rendering. Keep in mind, that's the *minimal* amount of expansion needed in order to support an extra state. If we also process events for each state individually or perform some kind of additional logic like *late updating*, that's four switch statements, one extra switch branch for each state and four extra methods that have to be implemented and added to the branches.

Next, consider state transitions. If, for whatever reason, you want to render two states at the same time for a short while, this entire approach collapses. It is possible to still somehow string that functionality together by tying up a bunch of flags or creating combination states as follows:

```
enum StateType{
    Intro = 1, Intro_MainMenu, MainMenu, Game, MainMenu_Game
    Paused, GameOver, MainMenu_GameOver, Credits, MainMenu_Credits
    ...
    // Crying in the corner.
};
```

This just keeps getting messier by the minute, and we haven't even got to expand our already large switch statements yet, let alone implement all the states we want!

If you still aren't thinking about migrating to a different tactic by now, consider this one final point: resources. If you're keeping all of the data from all of the possible states a game might have loaded at the same time, you may have quite a bit of a problem on your hands from the point of efficiency. You may dynamically allocate classes that represent certain states and check for when they're not in use anymore somehow to de-allocate them, however that's additional clutter in your already mostly unreadable code-base, and since you're already thinking of using classes, why not do it better?

Introducing the state pattern

All of the problems mentioned previously can be avoided after some careful white-boarding and consideration. The possibility was brought up earlier of different game states simply being localized to their own classes. All of these classes will share the same methods for being updated and rendered, which makes **inheritance** the word of the hour. Let's take a look at our base state header:

```
class StateManager;

class BaseState{
    friend class StateManager;
public:
    BaseState(StateManager* l_stateManager)
        :m_stateMgr(l_stateManager),m_transparent(false),
        m_transcendent(false){}
    virtual ~BaseState(){}

    virtual void OnCreate() = 0;
    virtual void OnDestroy() = 0;

    virtual void Activate() = 0;
    virtual void Deactivate() = 0;

    virtual void Update(const sf::Time& l_time) = 0;
    virtual void Draw() = 0;

    void SetTransparent(const bool& l_transparent){
        m_transparent = l_transparent;
    }
    bool IsTransparent()const{ return m_transparent; }
    void SetTranscendent(const bool& l_transcendence){
        m_transcendent = l_transcendence;
    }
}
```

```

    bool IsTranscendent()const{ return m_transcendent; }
    StateManager* GetStateManager(){ return m_stateMgr; }
protected:
    StateManager* m_stateMgr;
    bool m_transparent;
    bool m_transcendent;
};

```

First, you'll notice we're using a *forward declaration* of the `StateManager` class. The base class doesn't really need to know anything about the way our state manager will be implemented, only that it needs to keep a pointer to it. This is also done in order to avoid *recursive definitions*, because the `StateManager` class header needs to include the `BaseState` class header.

Since we want to enforce the use of the same methods throughout all states, we make them *purely virtual*, which means that the class inheriting from `BaseState` has to implement each and every one of them in order for the project to compile. The methods that any derived class has to implement consist of `OnCreate` and `OnDestroy`, which get invoked when the state is created and pushed on the stack, and later removed from the stack, `Activate` and `Deactivate`, which are called once a state is moved to the top of the stack as well as when it gets removed from the top position, and lastly, `Update` and `Draw`, which are used for updating the state and drawing its contents.

One last thing to note about this class is that it has a pair of flags: `m_transparent` and `m_transcendent`. These flags indicate if this state also needs to render or update a state that came before it. This eliminates the need for countless enumerations of different transitions between states and can be done automatically without any additional expansion.

Defining common types

One thing that we're definitely going to keep from the previous example is the enumeration table of the state types:

```

enum class StateType{
    Intro = 1, MainMenu, Game, Paused, GameOver, Credits
};

```

Having the state types enumerated is convenient and it helps with automating the state creation, as you will see later on.

Another common type we need to keep around is the device context we'll be using with our states. Don't be confused by the name, it simply means having a pointer to some of our most commonly used classes or "devices." Because there's more than one, it's quite useful to define a simple structure that will keep around pointers to the main window class and the event manager:

```
struct SharedContext{
    SharedContext():m_wind(nullptr),m_eventManager(nullptr){}
    Window* m_wind;
    EventManager* m_eventManager;
};
```

This can and will be expanded later when needed, in order to hold information about the player and other assistant classes that deal with resource allocation, sound and networking.

The state manager class

Now that we have our helper structures set up, let's actually define the types that will be used to hold information in our state manager class. As always, we will be using type definitions, the beauty of which is the fact that they reduce the amount of code you have to change in a case of modifying something about the type definition. Let's take a look at the state container type first:

```
using StateContainer = std::vector<
    std::pair<StateType, BaseState*>>;
```

Once again, we're using a vector. The element type is a pair of our state type and a pointer to a `BaseState` type object. You might be wondering why a map isn't a better choice, and the answer depends on your idea of implementation, however, one major factor is that a map doesn't keep a stack-like order in the container, which is important if we want our state manager to work correctly.

One of the design decisions in the state manager class also requires a container of state types, so let's define that:

```
using TypeContainer = std::vector<StateType>;
```

As you can see, it's simply a vector of the `StateType` enumeration types.

The last type we need to define is a container for custom functions that will serve as a way of automatically producing objects of different types derived from the `BaseState` class:

```
using StateFactory = std::unordered_map<
    StateType, std::function<BaseState*(void)>>;
```

We're using an unordered map here in order to map a specific state type to a specific function that will generate that type. If that sounds confusing now, be patient. It will be covered more thoroughly when we actually use it.

Defining the state manager class

All the individual bits and pieces we needed to actually bang out a header for the state manager class are now present, so let's write it:

```
class StateManager{
public:
    StateManager(SharedContext* l_shared);
    ~StateManager();

    void Update(const sf::Time& l_time);
    void Draw();

    void ProcessRequests();

    SharedContext* GetContext();
    bool HasState(const StateType& l_type);

    void SwitchTo(const StateType& l_type);
    void Remove(const StateType& l_type);
private:
    // Methods.
    void CreateState(const StateType& l_type);
    void RemoveState(const StateType& l_type);

    template<class T>
    void RegisterState(const StateType& l_type){...}

    // Members.
    SharedContext* m_shared;
    StateContainer m_states;
    TypeContainer m_toRemove;
    StateFactory m_stateFactory;
};
```

The constructor takes in a pointer to the SharedContext type we talked about earlier, which will be created in our main Game class. Predictably enough, the state manager also employs the use of Update and Draw methods, because it will be operated by the Game class, and it's nice to keep the interface familiar. For convenience sake, it offers helper methods for obtaining the context as well as determining if it currently has a certain state on the stack.

Concluding the public methods, we have `SwitchTo`, which takes in a state type and changes the current state to one that corresponds to said type, and `Remove`, for removing a state from the state stack by its type.

If you looked at the class definition from top to bottom, you may have noticed that we have a `TypeContainer` member called `m_toRemove`. In order to ensure smooth and error-free transitions, we cannot simply delete any state we want from the state container at any time. A simple solution here is keeping track of the state types we want to remove and only removing them when they're no longer being used, which is what the `ProcessRequests` method does. It is called last in the game loop, which ensures that the states in the `m_toRemove` container are no longer in use.

Let's continue with the more advanced private methods and implementation of our state manager class in the next section.

Implementing the state manager

In order to maintain the automated approach of creating our states on the heap, we must have some way of defining how they're created. The `m_stateFactory` member is a map that links a state type to a `std::function` type, which we can be set to hold a body of a function through use of the lambda expression:

```
template<class T>
void RegisterState(const StateType& l_type) {
    m_stateFactory[l_type] = [this]() -> BaseState*
    {
        return new T(this);
    };
}
```

The code above maps the type `l_type` in the `m_stateFactory` map to a function that simply returns a pointer to newly allocated memory. We're using templates here in order to reduce the amount of code. Because each state requires a pointer to the `StateManager` class in its constructor, we pass the *this pointer* in. We can now register different states like so:

```
StateManager::StateManager(SharedContext* l_shared)
: m_shared(l_shared)
{
    RegisterState<State_Intro>(StateType::Intro);
    RegisterState<State_MainMenu>(StateType::MainMenu);
    RegisterState<State_Game>(StateType::Game);
    RegisterState<State_Paused>(StateType::Paused);
}
```

It's time to begin implementing the rest of the class now. Let's take a look at the destructor:

```
StateManager::~StateManager() {
    for (auto &itr : m_states) {
        itr.second->OnDestroy();
        delete itr.second;
    }
}
```

Because we localize all the dynamic memory allocation of any states to this class, it's imperative that we also free the memory appropriately. Iterating over all the states and deleting the second value of the pair which makes up the element does just that.

Next, let's take a look at how to implement the draw method:

```
void StateManager::Draw() {
    if (m_states.empty()) { return; }
    if (m_states.back().second->IsTransparent()
        && m_states.size() > 1)
    {
        auto itr = m_states.end();
        while (itr != m_states.begin()) {
            if (itr != m_states.end()) {
                if (!itr->second->IsTransparent()) {
                    break;
                }
            }
            --itr;
        }
        for (; itr != m_states.end(); ++itr) {
            itr->second->Draw();
        }
    } else {
        m_states.back().second->Draw();
    }
}
```

First, just like the `Update` method, we check if the state container has *at least one* state. If it does, we check the most recently added one's **transparency flag**, as well as if there's more than one state on the stack, otherwise the transparency would be useless. If there's only one state on the stack or if the current state isn't transparent, we simply invoke its `Draw` method. Otherwise, things get a little bit more interesting.

In order to correctly render transparent states, we must call their respective `Draw` methods in a correct order, where the latest state on the stack is drawn on screen last. To do that, it's necessary to iterate through the state vector *backwards* until a state is found that is either not transparent or is the first state on the stack, which is what the `while` loop does. After such state is found, the `Draw` calls of all states from and including the one found, up to the very last one are invoked in the `for` loop. This effectively renders multiple states at once in correct order.

A fairly similar procedure is followed when updating states:

```
void StateManager::Update(const sf::Time& l_time){
    if (m_states.empty()){ return; }
    if (m_states.back().second->IsTranscendent()
        && m_states.size() > 1)
    {
        auto itr = m_states.end();
        while (itr != m_states.begin()){
            if (itr != m_states.end()){
                if (!itr->second->IsTranscendent()){
                    break;
                }
            }
            --itr;
        }
        for (; itr != m_states.end(); ++itr){
            itr->second->Update(l_time);
        }
    } else {
        m_states.back().second->Update(l_time);
    }
}
```

The state's *transcendence* flag is checked first, in order to determine whether the top state allows others to be updated. The state or states that need to get updated then have their `Update` methods invoked with the elapsed time passed in as the argument, more commonly known as **delta time**.

As always, we need to define some helper methods for a class to be truly flexible and useful:

```
SharedContext* StateManager::GetContext(){ return m_shared; }

bool StateManager::HasState(const StateType& l_type){
    for (auto itr = m_states.begin();
        itr != m_states.end(); ++itr)
```

```

    {
        if (itr->first == l_type){
            auto removed = std::find(m_toRemove.begin(),
                                    m_toRemove.end(), l_type);
            if (removed == m_toRemove.end()){ return true; }
            return false;
        }
    }
    return false;
}

```

The first method of obtaining the context is pretty straightforward. All it does is return a pointer to the `m_shared` member. The second method simply iterates over the `m_states` container until it finds a state with the type `l_type` and returns `true`. If it doesn't find such state, or if the state is found but it's about to be removed, it returns `false`. This gives us a way to check if a certain state is on the stack.

Having a way to remove a state is just as necessary as having a way of adding one. Let's implement the public method `Remove`:

```

void StateManager::Remove(const StateType& l_type) {
    m_toRemove.push_back(l_type);
}

```

This method pushes back a state type into the `m_toRemove` vector for later removal, which is then processed by this method:

```

void StateManager::ProcessRequests() {
    while (m_toRemove.begin() != m_toRemove.end()) {
        RemoveState(*m_toRemove.begin());
        m_toRemove.erase(m_toRemove.begin());
    }
}

```

The last method of this class that ever gets called, `ProcessRequests`, simply iterates over the `m_toRemove` vector and invokes a private method `RemoveState` which takes care of actual resource de-allocation. It then removes the element, ensuring the container is cleared.

Being able to change the current state is of paramount importance, which is what the `SwitchTo` method takes care of:

```

void StateManager::SwitchTo(const StateType& l_type) {
    m_shared->m_eventManager->SetCurrentState(l_type);
    for (auto itr = m_states.begin();
         itr != m_states.end(); ++itr)

```

```
{
    if (itr->first == l_type){
        m_states.back().second->Deactivate();
        StateType tmp_type = itr->first;
        BaseState* tmp_state = itr->second;
        m_states.erase(itr);
        m_states.emplace_back(tmp_type, tmp_state);
        tmp_state->Activate();
        return;
    }
}

// State with l_type wasn't found.
if (!m_states.empty()){ m_states.back().second->Deactivate(); }
CreateState(l_type);
m_states.back().second->Activate();
}
```

First, you will notice that we access the event manager through our shared context and call a method `SetCurrentState`. We haven't yet gotten around to adding it, however it will be covered shortly. What it does is it simply modifies an internal data member of the event manager class, which keeps track of which state the game is in.

Next, we must find the state with the type we want to switch to, so we iterate over the state vector. If we have a match, the current state that's about to be pushed back has its `Deactivate` method called to perform whatever functionality it has to, in case the state cares about when it gets moved down. Then, we create two temporary variables to hold the state type and the pointer to a state object, so we don't lose that information when the element we're interested in is removed from the vector by calling `erase`. After doing that, all the *iterators* to the state container are invalidated, but it doesn't matter in our case, because we no longer need any. Moving the desired state is now as simple as pushing back another element onto the vector and passing in our temporary variables. Then, we call the `Activate` method of the state that just got moved in case it has any logic that is needed to be performed at that time.

If the state with `l_type` isn't found, creating one is necessary. First, however, it's important to check if there's at least one state for which to call the `Deactivate` method, and call it, if there is one. After invoking a private method `CreateState` and passing in the state type, we grab the element from the state vector that was added most recently by `CreateState`, and call `Activate`.

It's time to see what exactly goes into creating a state:

```
void StateManager::CreateState(const StateType& l_type) {
    auto newState = m_stateFactory.find(l_type);
    if (newState == m_stateFactory.end()) { return; }
    BaseState* state = newState->second();
    m_states.emplace_back(l_type, state);
    state->OnCreate();
}
```

A state factory iterator gets created and checked for matching the iterator returned by the `end()` method of `std::unordered_map`, allowing us to make sure a state with such type can be created. If it can, a pointer of type `BaseState`, called `state` is created. It catches the return result of our iterator's second value getting invoked as a function, which if you remember was the `std::function` type and returns a pointer to a newly created state class. This is how we put the previously mentioned "factory" to work. After retrieving a pointer to the newly allocated memory for a state, we simply push it back onto the state vector and call `OnCreate` for the state to do its internal logic regarding being freshly created.

How do we go about removing a state? Let's take a look:

```
void StateManager::RemoveState(const StateType& l_type) {
    for (auto itr = m_states.begin();
         itr != m_states.end(); ++itr)
    {
        if (itr->first == l_type) {
            itr->second->OnDestroy();
            delete itr->second;
            m_states.erase(itr);
            return;
        }
    }
}
```

As always when dealing with `std::vector` types, we iterate over it until a match is found. Removing the actual state begins by calling the `OnDestroy` method of said state, again, just so it can perform whatever logic it needs in order to be ready for removal. Then we simply de-allocate the memory by using the `delete` keyword. Finally, we erase the element from the state vector and return from the method.

Improving the Event Manager class

Having different states in a game will, without a shadow of a doubt, create situations where the same key or event will be needed by at least two of the states. Let's say we have a menu, where navigation is done by pressing the arrow keys. That's all fine, but what if the game-play state also registers the use of arrow keys and sets up its own callbacks? The very best case scenario is that callbacks from all states will be invoked at the same time and create weird behavior. Things get worse, however, when you have function pointers to methods that are no longer in memory, especially since nobody likes application crashes. A simple way of dealing with this problem is grouping the callbacks together by state and only invoking them if the current state is that of a callback. This obviously means some re-definition of the types being dealt with:

```
using CallbackContainer = std::unordered_map<
    std::string, std::function<void(EventDetails*)>>;
enum class StateType;
using Callbacks = std::unordered_map<
    StateType, CallbackContainer>;
```

Things are getting a little bit more complicated now. What used to be the `Callback` definition is now renamed `CallbackContainer`. We only want one of those per state, so it means having to use another map, which is where the new `Callback` definition comes in. It maps a state type to a `CallbackContainer` type, so that we can have only one `CallbackContainer` per state in addition to only one callback function per name.

Despite these changes, the declaration for `m_callbacks` in the event manager header remains the same:

```
Callbacks m_callbacks;
```

There is one minor addition to the class data member list, and that is the current state:

```
StateType m_currentState;
```

What does change, however, are the methods for adding, removing and utilizing callbacks. Let's adapt the `AddCallback` method to these changes:

```
template<class T>
bool AddCallback(StateType l_state, const std::string& l_name,
    void(T::*l_func)(EventDetails*), T* l_instance)
{
    auto itr = m_callbacks.emplace(
        l_state, CallbackContainer()).first;
```

```

    auto temp = std::bind(l_func, l_instance,
        std::placeholders::_1);
    return itr->second.emplace(l_name, temp).second;
}

```

The first thing to note is that we have a new argument `l_state` in the method's footprint. Next, we attempt to insert a new element to the `m_callbacks` map, pairing together the state argument and a new `CallbackContainer`. Since a map can only have one element with a specific index, in this case it's the state type, the `emplace` method always returns a pair of elements, the first of which is an iterator. If the insertion succeeded, the iterator points to the element that was newly created. On the other hand, if an element with a specified index already existed, the iterator points to that element instead. This is a good strategy to use, because we need that iterator no matter what, and if there is no element with the index we specified, we're going to want to insert one.

After the function binding, which remains unchanged, we need to insert the actual callback into the `CallbackContainer` type, which is the second value in the pair that makes up the `m_callbacks` elements. The second value of a pair that gets returned by the insert method of a map is a Boolean that represents the success of an insertion, and that's what gets returned for error checking.

Now let's take a look at revising the removal of callbacks:

```

bool RemoveCallback(StateType l_state, const std::string& l_name){
    auto itr = m_callbacks.find(l_state);
    if (itr == m_callbacks.end()){ return false; }
    auto itr2 = itr->second.find(l_name);
    if (itr2 == itr->second.end()){ return false; }
    itr->second.erase(l_name);
    return true;
}

```

This one's fairly simple. All we do is use the `find` method twice instead of once. First, we find the state pair in the first map, then we `erase` the actual callback by its name in the second map, just like before.

The last part of making this work just the way we want is fixing the way callback functions are actually called. Due to the type definitions that got changed, the way we invoke callbacks is also slightly different:

```

void EventManager::Update() {
    ...
    if (bind->m_events.size() == bind->c){
        auto stateCallbacks = m_callbacks.find(m_currentState);

```

```
    auto otherCallbacks = m_callbacks.find(StateType(0));

    if (stateCallbacks != m_callbacks.end()){
        auto callItr = stateCallbacks->second.find(bind->m_name);
        if (callItr != stateCallbacks->second.end()){
            // Pass in information about events.
            callItr->second(&bind->m_details);
        }
    }

    if (otherCallbacks != m_callbacks.end()){
        auto callItr = otherCallbacks->second.find(bind->m_name);
        if (callItr != otherCallbacks->second.end()){
            // Pass in information about events.
            callItr->second(&bind->m_details);
        }
    }
}
...
}
```

The main difference here is that we have two states for which callbacks get checked now, not just one: `stateCallbacks` and `otherCallbacks`. The former is quite obvious, we're simply using `find` to obtain the map of all callbacks for the current state. The latter, however, passes in a state type value of 0, which isn't a valid state type, since the enumeration starts at 1. This is done because even in the case of having multiple states in a game, we still want to process global callbacks for the `Window` class, as well as other classes that extend beyond the scope of simple states and persist all the way throughout the life of an application. Anything with the state type 0 will be invoked regardless of which state we're in.

The rest is fairly straightforward. Just like before, we're using the `find` method of the second value in the iterator that gets returned from the first search, which is our actual callback map. If a match is found, the function gets invoked.

One last thing we want to do here is modify the `keys.cfg` file to hold some extra keys for us in order to use them later:

```
Window_close 0:0
Fullscreen_toggle 5:89
Intro_Continue 5:57
Mouse_Left 9:0
Key_Escape 5:36
Key_P 5:15
```

The `Intro_Continue` binding represents a Spacebar "key down" event, `Mouse_Left` is the mouse left click event, `Key_Escape` is bound to the *ESC* "key down" event, and lastly, `Key_P` represents the letter *P* "key down" event.

Incorporating the state manager

While it's not quite time for fanfare, excitement is definitely in order because we can finally put our brand new `StateManager` class to work! The `Game` class header modification is a good start:

```
...
#include "StateManager.h"
...
class Game{
public:
    ...
    void LateUpdate();
private:
    ...
    StateManager m_stateManager;
};
```

Sticking a new data member to the `Game` class and adding a new method for late updating are all the adjustments that need to be made in the header. Let's adjust the `Game` constructor to initialize the state manager:

```
Game::Game(): m_window(sf::Vector2u(800, 600)),
    m_stateManager(&m_context)
{
    ...
    m_context.m_wind = &m_window;
    m_context.m_eventManager = m_window.GetEventManager();
    m_stateManager.SwitchTo(StateType::Intro);
}
```

Naturally, the first thing we do is create the context that will be used by all of the states and pass it into the constructor of the state manager. We then begin the "domino effect" by switching to the introduction state, which will in due time switch to other states and force the flow of the application.

Lastly, let's adjust the three most important methods of the Game class:

```
void Game::Update() {
    m_window.Update();
    m_stateManager.Update(m_elapsed);
}
void Game::Render() {
    m_window.BeginDraw();
    m_stateManager.Draw();
    m_window.EndDraw();
}
void Game::LateUpdate() {
    m_stateManager.ProcessRequests();
    RestartClock();
}
```

That's about as straightforward as it can be. One thing to note is that the `RestartClock` method is now called by the `LateUpdate`, which means we have to adjust the `main.cpp` file as follows:

```
#include "Game.h"

void main(int argc, void** argv){
    // Program entry point.
    Game game;
    while(!game.GetWindow()->IsDone()){
        game.Update();
        game.Render();
        game.LateUpdate();
    }
}
```

Everything seems to be in order now. Compiling and launching the application should give you a very impressive black screen. Hoorah! Let's actually create some states for the game in order to honor the work that was put into this.

Creating the intro state

It seems rather fitting to start with the intro state, in turn giving the state manager a bit of an introduction at the same time. As always, a good place to start is with the header file, so let's get going:

```
class State_Intro : public BaseState{
public:
    ...
}
```

```

        void Continue(EventDetails* l_details);
    private:
        sf::Texture m_introTexture;
        sf::Sprite m_introSprite;
        sf::Text m_text;
        float m_timePassed;
    };

```

The `State_Intro` class, just like all the other state classes we'll build, inherits from the `BaseState` class. All of the purely virtual methods of the base class have to be implemented here. In addition to that, we have a unique method named `Continue` and some private data members that will be used in this state. Predictably enough, we will be rendering a sprite on screen, as well as some text. The floating point data member on the very bottom will be used to keep track of how much time we have spent in this state, in order to present the user with the ability to hit the Spacebar key after a certain interval to proceed into the main menu. The `Continue` method is responsible for handling that transition.

Implementing the intro state

We are getting close to finishing our first functional state! All that needs to be finished now is the actual implementation of the methods declared in the header file, and we're golden. Let's begin by including the header file of our class in `State_Intro.cpp`:

```

#include "State_Intro.h"
#include "StateManager.h"

```

Note the second line. Because the `StateManager` class is forwardly declared in the `BaseState` header, we *must* include the state manager header in the implementation file. This is true for any state we build in the future, including this one.

We will never use constructors and destructors of our states to initialize or allocate anything and instead rely on the `OnCreate` and `OnDestroy` methods in order to retain maximum control of when the resource allocation and de-allocation actually happens:

```

void State_Intro::OnCreate() {
    m_timePassed = 0.0f;

    sf::Vector2u windowSize = m_stateMgr->GetContext()->
        m_wind->GetRenderWindow()->getSize();

    m_introTexture.loadFromFile("intro.png");
    m_introSprite.setTexture(m_introTexture);
}

```

```
m_introSprite.setOrigin(m_introTexture.getSize().x / 2.0f,
    m_introTexture.getSize().y / 2.0f);

m_introSprite.setPosition(windowSize.x / 2.0f, 0);

m_font.loadFromFile("arial.ttf");
m_text.setFont(m_font);
m_text.setString({ "Press SPACE to continue" });
m_text.setCharacterSize(15);
sf::FloatRect textRect = m_text.getLocalBounds();
m_text.setOrigin(textRect.left + textRect.width / 2.0f,
    textRect.top + textRect.height / 2.0f);
m_text.setPosition(windowSize.x / 2.0f, windowSize.y / 2.0f);

EventManager* evMgr = m_stateMgr->
    GetContext() -> m_eventManager;
evMgr->AddCallback(StateType::Intro, "Intro_Continue",
    &State_Intro::Continue, this);
}
```

There's quite a bit of code, however, only a tiny portion of it is new to us at this point. First, we must initialize our data member `m_timePassed` to zero. Next, we obtain the shared context through the use of the state manager pointer from the base class, and use it to obtain the current window size.

In order to position the `m_text` right in the middle of the screen, we set its origin to be the absolute center first, which is done by first obtaining a `sf::FloatRect` data type by calling the `getLocalBounds` method of our `sf::text` object. The left and top values of the `sf::FloatRect` represent the top left corner of the text, which can be used to calculate the center by adding half of the rectangle size to it.



If any changes are made to the character size, the string or to the font that the `sf::text` object is using, the origin has to be re-calculated, because the physical dimensions of the local boundary rectangle are changed too.

The basic idea of this intro state is to have a sprite come down from the top of the screen to the middle. After five seconds have passed, some text will appear underneath the sprite notifying the user that they can hit the Spacebar in order to proceed to the main menu. This is the texture we will be using for the descending sprite:



The last thing we need to do is to bind the Spacebar key to the `Continue` method of our intro class. We do that by obtaining the event manager instance through the shared context and setting up the callback, pretty much as we did in the previous chapter, except this time we need an additional argument: the state type.

Even though this class doesn't allocate any memory, it's still important it removes its callback when removed, which can be done here:

```
void State_Intro::OnDestroy() {
    EventManager* evMgr = m_stateMgr->
        GetContext()->m_eventManager;
    evMgr->RemoveCallback(StateType::Intro, "Intro_Continue");
}
```

Just like the `AddCallback` method, removal of callbacks also requires a state type as its first argument.

Because we're dealing with time and movement here, updating this state will be necessary:

```
void State_Intro::Update(const sf::Time& l_time){
    if(m_timePassed < 5.0f){ // Less than five seconds.
        m_timePassed += l_time.asSeconds();
        m_introSprite.setPosition(m_introSprite.getPosition().x,
            m_introSprite.getPosition().y + (48 * l_time.asSeconds()));
    }
}
```

Seeing how it's only desired for the sprite to be moving until it reaches the middle, a five second window is defined. If the total time passed is less than that, we add the delta time argument to it for the next iteration and move the sprite by a set number of pixels per second in the y direction, while keeping x the same. This guarantees vertical movement, which is, of course, completely useless, unless we draw everything:

```
void State_Intro::Draw(){
    sf::RenderWindow* window = m_stateMgr->
        GetContext()->m_wind->GetRenderWindow();
```

```
        window->draw(m_introSprite);  
        if(m_timePassed >= 5.0f){  
            window->draw(m_text);  
        }  
    }
```

After obtaining a pointer to a window through the shared context, we draw the sprite on screen. If more than five seconds have passed, we also draw the text, which notifies the player about the possibility of continuing past the intro state, the final piece of the puzzle:

```
void State_Intro::Continue() {  
    if(m_timePassed >= 5.0f) {  
        m_stateMgr->SwitchTo(StateType::MainMenu);  
        m_stateMgr->Remove(StateType::Intro);  
    }  
}
```

Once again, we check if enough time has passed to continue past this state. The actual switching happens when the `SwitchTo` method is called. Because we won't need the introduction state on the stack anymore, it removes itself in the next line.

Although we won't be needing the last two methods, we still need to implement empty versions of them, like so:

```
void State_Intro::Activate() {}  
void State_Intro::Deactivate() {}
```

Now it's time to sound the fanfares! Our first state's done and is ready for use. Building and launching your application should leave you with something like this:



As illustrated above, the sprite descends all the way to the middle of the screen and displays the message about continuing underneath after five seconds. Upon hitting Spacebar you will find yourself in a black window because we haven't implemented the main menu state yet.

From this point on, all the repetitive code will be left out. For complete source code, please take a look at the source files of this chapter.

The main menu state

The main menu of any game out there is a major vein in terms of application flow, even though it's mostly overlooked. It's time we took a stab at building one, albeit a very simplistic version, starting as always with the header file:

```
class State_MainMenu : public BaseState{
public:
    ...
    void MouseClick(EventDetails* l_details);
private:
    sf::Text m_text;

    sf::Vector2f m_buttonSize;
    sf::Vector2f m_buttonPos;
    unsigned int m_buttonPadding;

    sf::RectangleShape m_rects[3];
    sf::Text m_labels[3];
};
```

The unique method to this class is the `MouseClick`. Since we're dealing with a menu here, predictably enough it will be used to process mouse input. For private data members, we have a text variable for the title, size, position and padding size variables for buttons, drawable rectangles for buttons and text variables for button labels. Let's throw it all together:

```
void State_MainMenu::OnCreate(){
    m_font.loadFromFile("arial.ttf");
    m_text.setFont(m_font);
    m_text.setString(sf::String("MAIN MENU:"));
    m_text.setCharacterSize(18);

    sf::FloatRect textRect = m_text.getLocalBounds();
    m_text.setOrigin(textRect.left + textRect.width / 2.0f,
        textRect.top + textRect.height / 2.0f);
```

```
m_text.setPosition(400,100);

m_buttonSize = sf::Vector2f(300.0f,32.0f);
m_buttonPos = sf::Vector2f(400,200);
m_buttonPadding = 4; // 4px.

std::string str[3];
str[0] = "PLAY";
str[1] = "CREDITS";
str[2] = "EXIT";

for(int i = 0; i < 3; ++i){
    sf::Vector2f buttonPosition(m_buttonPos.x,m_buttonPos.y +
        (i * (m_buttonSize.y + m_buttonPadding)));
    m_rects[i].setSize(m_buttonSize);
    m_rects[i].setFillColor(sf::Color::Red);

    m_rects[i].setOrigin(m_buttonSize.x / 2.0f,
        m_buttonSize.y / 2.0f);
    m_rects[i].setPosition(buttonPosition);

    m_labels[i].setFont(m_font);
    m_labels[i].setString(sf::String(str[i]));
    m_labels[i].setCharacterSize(12);

    sf::FloatRect rect = m_labels[i].getLocalBounds();
    m_labels[i].setOrigin(rect.left + rect.width / 2.0f,
        rect.top + rect.height / 2.0f);

    m_labels[i].setPosition(buttonPosition);
}

EventManager* evMgr = m_stateMgr->
    GetContext()->m_eventManager;
evMgr->AddCallback(StateType::MainMenu,"Mouse_Left",
    &State_MainMenu::MouseClicked,this);
}
```

In the method above, all of the graphical elements get set up. The text data members get defined, origins are set up, and the labels for individual buttons get named. Lastly, the callback for the mouse left click gets set up. This is by no means a sophisticated GUI system. A more robust way of actually designing one will be covered in later chapters, however, this will suit our needs for now.

When the state gets destroyed, we need to remove its callbacks, as mentioned before:

```
void State_MainMenu::OnDestroy() {
    EventManager* evMgr = m_stateMgr->
        GetContext()->m_eventManager;
    evMgr->RemoveCallback(StateType::MainMenu, "Mouse_Left");
}
```

Upon the state getting activated, we need to check if the main game-play state exists on the state stack in order to adjust the "play" button to instead say "resume":

```
void State_MainMenu::Activate() {
    if(m_stateMgr->HasState(StateType::Game)
        && m_labels[0].getString() == "PLAY")
    {
        m_labels[0].setString(sf::String("RESUME"));
        sf::FloatRect rect = m_labels[0].getLocalBounds();
        m_labels[0].setOrigin(rect.left + rect.width / 2.0f,
            rect.top + rect.height / 2.0f);
    }
}
```



The text origin has to be recalculated again because the dimensions of the `sf::drawable` object are now different.

The `MouseClicked` method can be implemented as follows:

```
void State_MainMenu::MouseClicked(EventDetails* l_details) {
    sf::Vector2i mousePos = l_details->m_mouse;

    float halfX = m_buttonSize.x / 2.0f;
    float halfY = m_buttonSize.y / 2.0f;
    for(int i = 0; i < 3; ++i) {
        if(mousePos.x >= m_rects[i].getPosition().x - halfX &&
            mousePos.x <= m_rects[i].getPosition().x + halfX &&
            mousePos.y >= m_rects[i].getPosition().y - halfY &&
            mousePos.y <= m_rects[i].getPosition().y + halfY)
        {
            if(i == 0) {
                m_stateMgr->SwitchTo(StateType::Game);
            } else if(i == 1) {
                // Credits state.
            } else if(i == 2) {
```

```
        m_stateMgr->GetContext()->m_wind->Close();
    }
}
}
```

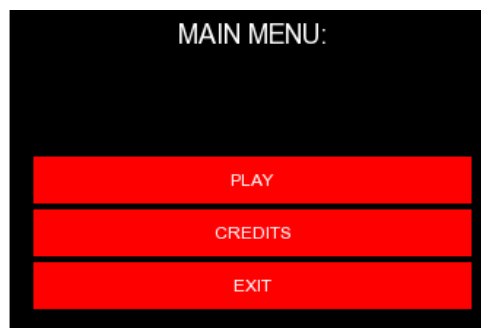
First, we obtain the mouse position from the event information structure, which gets passed in as the argument. Then we set up some local floating point type variables that will be used to check the boundaries of the buttons and begin looping over all the buttons. Because the origins of every button are set to the absolute middle, we must adjust the position according to that when checking if the mouse position is within the rectangle. If we have a mouse to button collision, an if-else statement checks which ID has collided and performs an action accordingly. In the case of the "play" button being pressed, we switch to the game state. If the exit button is pressed, we invoke the `Window::Close` method through the shared context.

Finally, let's draw the main menu:

```
void State_MainMenu::Draw() {
    sf::RenderWindow* window = m_stateMgr->GetContext()->
        m_wind->GetRenderWindow();
    window->draw(m_text);
    for(int i = 0; i < 3; ++i) {
        window->draw(m_rects[i]);
        window->draw(m_labels[i]);
    }
}
```

After obtaining the render window pointer through the shared context, drawing the entire menu is as easy as iterating a few times to draw a button and a label.

Upon successful compilation and execution, we're again presented with the intro screen. When hitting spacebar, a main menu opens, looking something like this:



It's not the prettiest sight in the world, but it gets the job done. Clicking the **PLAY** button once again leaves us with a black screen, while hitting **EXIT** closes the application. Neat!

A sample game state

Just to demonstrate the full use of our system, let's get something bouncing on the screen that will demonstrate switching between the menu, game, and paused states. For testing purposes, a bouncing mushroom from previous chapters will more than suffice. We also need methods for switching to the menu state, as well as the paused state. Knowing that, let's bang out the header for the game-play state:

```
class State_Game : public BaseState{
public:
    ...
    void MainMenu(EventDetails* l_details);
    void Pause(EventDetails* l_details);
private:
    sf::Texture m_texture;
    sf::Sprite m_sprite;
    sf::Vector2f m_increment;
};
```

We begin, like many other times, with resource allocation and set up of data members in the `OnCreate` method:

```
void State_Game::OnCreate() {
    m_texture.loadFromFile("Mushroom.png");
    m_sprite.setTexture(m_texture);
    m_sprite.setPosition(0,0);
    m_increment = sf::Vector2f(400.0f,400.0f);

    EventManager* evMgr = m_stateMgr->
        GetContext()->m_eventManager;
    evMgr->AddCallback(StateType::Game, "Key_Escape",
        &State_Game::MainMenu, this);
    evMgr->AddCallback(StateType::Game, "Key_P",
        &State_Game::Pause, this);
}
```

After loading the texture and binding the sprite to it, we set up its position, define the increment vector, much like before, and add callbacks to our extra two methods for switching to different states. Of course, we need to remove them upon destruction of the state, like so:

```
void State_Game::OnDestroy() {
    EventManager* evMgr = m_stateMgr->
        GetContext()->m_eventManager;
    evMgr->RemoveCallback(StateType::GAME, "Key_Escape");
    evMgr->RemoveCallback(StateType::GAME, "Key_P");
}
```

The update method will hold the same code we've used previously:

```
void State_Game::Update(const sf::Time& l_time) {
    sf::Vector2u l_windSize = m_stateMgr->GetContext()->
        m_wind->GetWindowSize();
    sf::Vector2u l_textSize = m_texture.getSize();

    if((m_sprite.getPosition().x > l_windSize.x -
        l_textSize.x && m_increment.x > 0) ||
        (m_sprite.getPosition().x < 0 && m_increment.x < 0))
    {
        m_increment.x = -m_increment.x;
    }

    if((m_sprite.getPosition().y > l_windSize.y -
        l_textSize.y && m_increment.y > 0) ||
        (m_sprite.getPosition().y < 0 && m_increment.y < 0))
    {
        m_increment.y = -m_increment.y;
    }

    m_sprite.setPosition(m_sprite.getPosition().x +
        (m_increment.x * l_time.asSeconds()),
        m_sprite.getPosition().y +
        (m_increment.y * l_time.asSeconds()));
}
```

The sprite position gets checked, and if it is outside of the window boundaries, the increment vector on the appropriate axis gets inverted. Then, the sprite position is updated, taking into account the time passed between frames. It's as regular as clockwork. Let's draw the sprite on the screen:

```
void State_Game::Draw() {
    m_stateMgr->GetContext()->m_wind->
        GetRenderWindow()->draw(m_sprite);
}
```

Now let's implement the methods for switching states:

```
void State_Game::MainMenu(EventDetails* l_details) {
    m_stateMgr->SwitchTo(StateType::MAIN_MENU);
}

void State_Game::Pause(EventDetails* l_details) {
    m_stateMgr->SwitchTo(StateType::PAUSED);
}
```

Notice that the game state does not remove itself here, just like the main menu state. This means that it's still alive in memory and is waiting to be pushed back to the front of the vector to be updated and rendered again. This allows the user to pop back to the main menu and resume the game state at any time without losing progress.

Running the application now will transition us through the intro state into the main menu. Hitting the **PLAY** button will leave us with a bouncing mushroom, just like before:



Hitting the escape key now will bring you back to the main menu, at which point you can choose to click the **RESUME** button to pop back into the game state, or the **EXIT** button to quit the application. There's just one more state left to implement to fully showcase the abilities of this system!

The means to pause

One might simply consider navigating to the main menu from the game state as a way of putting the game on pause. While that's technically true, why not explore a second option, which looks much trendier than simply popping the main menu open? After writing so much code, we deserve a nice looking paused state:

```
class State_Paused : public BaseState{
public:
    ...
    void Unpause(EventDetails* l_details);
private:
```

```
        sf::Text m_text;  
        sf::RectangleShape m_rect;  
    };
```

This one is quite simple. Once more, we define an additional method, in this case `Unpause`, to switch to a different state. There's also only two data members used in order to draw the text "PAUSED" on screen, as well as a nice semi-transparent backdrop, represented by the `sf::RectangleShape`. Let's implement the `OnCreate` method for the last time in this chapter:

```
void State_Paused::OnCreate() {  
    SetTransparent(true); // Set our transparency flag.  
    m_font.loadFromFile("arial.ttf");  
    m_text.setFont(m_font);  
    m_text.setString(sf::String("PAUSED"));  
    m_text.setCharacterSize(14);  
    m_text.setStyle(sf::Text::Bold);  
  
    sf::Vector2u windowSize = m_stateMgr->  
        GetContext()->m_wind->GetRenderWindow()->getSize();  
  
    sf::FloatRect textRect = m_text.getLocalBounds();  
    m_text.setOrigin(textRect.left + textRect.width / 2.0f,  
        textRect.top + textRect.height / 2.0f);  
    m_text.setPosition(windowSize.x / 2.0f, windowSize.y / 2.0f);  
  
    m_rect.setSize(sf::Vector2f(windowSize));  
    m_rect.setPosition(0,0);  
    m_rect.setFillColor(sf::Color(0,0,0,150));  
  
    EventManager* evMgr = m_stateMgr->  
        GetContext()->m_eventManager;  
    evMgr->AddCallback(StateType::Paused, "Key_P",  
        &State_Paused::Unpause, this);  
}
```

A distinct difference here is the use of `m_transparent` flag, which is a protected data member of the `BaseState` class. Setting it to true means we're allowing the state manager to render the state directly behind this one on the state stack.

Besides that, we create a rectangle the size of the entire window and set its fill color to black with the alpha channel value of 150 out of the maximum 255. This makes it nice and translucent while darkening everything that's behind it.

The final part of the method above, quite like all the other ones, is adding the callback to the `Unpause` method. Upon destruction of this state, it needs to be removed like so:

```
void State_Paused::OnDestroy() {
    EventManager* evMgr = m_stateMgr->
        GetContext()->m_eventManager;
    evMgr->RemoveCallback(StateType::Paused, "Key_P");
}
```

Now let's draw the rectangle and text we created:

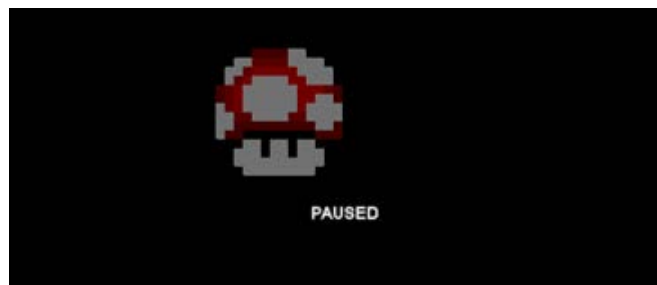
```
void State_Paused::Draw() {
    sf::RenderWindow* wind = m_stateMgr->
        GetContext()->m_wind->GetRenderWindow();
    wind->draw(m_rect);
    wind->draw(m_text);
}
```

Also, let's implement the `Unpause` method by simply switching to the game-play state:

```
void State_Paused::Unpause(EventDetails* l_details) {
    m_stateMgr->SwitchTo(StateType::Game);
}
```

Because the main game state is the only state that can be paused so far, simply switching back to it is sufficient.

Now, take a deep breath and compile the application again. Getting past the intro state, hitting the **PLAY** button in the main menu, and hitting the **P** key on your keyboard will effectively pause the game-play state and darken the screen subtly, while displaying the text **PAUSED** right in the middle, as shown here:



If you have come this far, congratulations! While this is by no means a finished product, it has come a long way from being a static, immovable class that can barely be controlled.

Common mistakes

A likely mistake that might be made when using this system is the absence of registration of newly added states. If you have built a state and it simply draws a black screen when you switch to it, chances are it was never registered in the constructor of `StateManager`.

The window not responding to the `F5` key being pressed or the close button being hit is a sign of the global callbacks not being set up right. In order to make sure a callback is invoked no matter which state you're in, it must be set up with the state type of 0, like so:

```
m_eventManager->AddCallback(StateType(0), "Fullscreen_toggle",
    &Window::ToggleFullscreen, this);
m_eventManager->AddCallback(StateType(0), "Window_close",
    &Window::Close, this);
```

Finally, remember that when the mouse position is retrieved in the main menu state, the coordinates stored inside the event are automatically relative to the window. Obtaining coordinates through `sf::Mouse::GetPosition` is not going to do the same, unless a reference to a `sf::Window` class is provided as an argument.

Summary

Upon this chapter concluding, you should have everything you need in your tool belt to fashion states that can be transparent, updated in groups, and supported by the rest of our codebase. There's no reason to stop there. Build it again, make it better, faster and implement different features that didn't get covered in this chapter. Expand it, crash it, fix it and learn from it. Nothing is ever good enough, so build onto the knowledge you've gained here.

A famous Chinese proverb states: "Life is like a game of chess, changing with each move".

While that analogy holds true, life can also be like a game with states. Breaking it down into smaller and more manageable parts makes it a whole lot easier to handle. Whether it is life imitating code or code imitating life is irrelevant. Great ideas come from different backgrounds coming together. Hopefully, by the end of this chapter you are taking off with not only the knowledge of simply how to build yet another manager, but also the wisdom to seek inspiration from every resource and idea available. There is no exclusive knowledge, only inclusive thinking. See you in the next chapter!